

1. (2.5pts) Considere uma ordenação topológica O para o grafo dirigido $G = (V, A)$. Considere também o grafo $G' = (V', A')$, no qual $V' = V \cup \{x\}$ e $A' = A \cup \{a\}$, $a \in \{(x, v), (v, x)\}$ para um vértice $v \in V$. Desenvolva um algoritmo eficiente (usando pseudocódigo ou alguma linguagem de programação) que receba G' , O , o vértice x e o arco a , depois, retorne uma ordenação topológica para G' sem ter que utilizar o algoritmo para **Ordenação Topológica** visitado na disciplina.

• Como x possui apenas uma única conexão, só é preciso garantir que essa posição respeite essa única conexão:

- Se arco $a = (x, v)$, x deve vir antes de $v \rightarrow$ colocar x imediatamente antes de v .
- Se arco $a = (v, x)$, vice-versa.

• a ordem relativa dos outros vértices continua a mesma

• entrada: grafo G' , $O = \langle v_1, v_2, \dots, v_k \rangle$, x , a

```

1. if  $a = (x, v)$  then
2.    $p \leftarrow 0$ 
3.   for each  $i \in \{1, 2, \dots, k\}$  do
4.     if  $v_i == v$  then
5.        $p \leftarrow i$ 
6.    $O' \leftarrow \langle v_1, \dots, v_{p-1}, x, v_p, \dots, v_k \rangle$ 
7. else
8.    $p \leftarrow 0$ 
9.   for each  $i \in \{1, 2, \dots, k\}$  do
10.    if  $v_i == v$  then
11.       $p \leftarrow i$ 
12.    $O' \leftarrow \langle v_1, \dots, v_p, x, v_{p+1}, \dots, v_k \rangle$ 
13. return  $O'$ 

```

$\rightarrow$ Se x aparece antes de v

$\rightarrow$ variável aux com valor 0

$\rightarrow$ p/ todos índices de O

$\left\{ \begin{array}{l} \text{Se o vértice atual é o vértice que procuramos,} \\ \text{p recebe o índice } i \end{array} \right.$

$\rightarrow$ copia todos elementos até o anterior a v , add x , copia o restante

$\rightarrow$ caso contrário, se x aparece depois de v

$O(|V|)$

$O(|V|)$

$O(|V|)$

$\hookrightarrow$ da 8) somar loop de p e fazer 'if $a = \dots$, else' no final

3. (2.5pts) Considere dois grafos não-dirigidos e ponderados $G_1 = (V_1, E_1, w_1)$ e $G_2 = (V_2, E_2, w_2)$, para os quais $V_1 \cap V_2 = \emptyset$. Considere também o grafo $G = (V_1 \cup V_2, E_1 \cup E_2 \cup \{\{x, y\}\}, w)$ no qual $x \in V_1$, $y \in V_2$, $w(\{u, v\}) = w_1(\{u, v\})$ se $\{u, v\} \in E_1$, $w(\{u, v\}) = w_2(\{u, v\})$ se $\{u, v\} \in E_2$ e $w(\{x, y\}) = \rho$. Sabendo que a árvore geradora mínima de G_1 é $A_1 \subseteq E_1$ e a árvore geradora mínima de G_2 é $A_2 \subseteq E_2$, é possível afirmar que $A_1 \cup A_2 \cup \{\{x, y\}\}$ é a **árvore geradora mínima** de G ? Justifique sua resposta.

• Vértices são todos os vértices; arestas são todas as arestas + $\{\{x, y\}\}$ ponte (conecta x de G_1 com y de G_2); arestas mantêm originais e $\{\{x, y\}\}$ tem peso ρ

• $A_1 \cup A_2 \cup \{\{x, y\}\}$ é AG válida?

$\hookrightarrow$ Conectividade: A_1 conecta G_1 e A_2 conecta G_2

• $\{\{x, y\}\}$ conecta G_1 e G_2 , logo todo grafo está conectado

$\hookrightarrow$ Acíclico: A_1 e A_2 não tem ciclos por definição

• El fechar ciclo entre 2 grafos isolados é necessário pelo menos 2 arestas (ida e volta) $\rightarrow$ só $\{\{x, y\}\}$ não faz isso

$\hookrightarrow$ nº arestas: A_1 tem $|V_1| - 1$ arestas; A_2 tem $|V_2| - 1$ arestas

• $(|V_1| - 1) + (|V_2| - 1) + 1 (\{\{x, y\}\}) = |V_1| + |V_2| - 1$

• Como vértices são disjuntos, nº de vértices é $|V_1| + |V_2|$, logo nº total de arestas $|V_1| + |V_2| - 1$

• é uma árvore geradora

• $A_1 \cup A_2 \cup \{\{x, y\}\}$ é mínima?

• Se grafo possui uma aresta que é a única ligação entre componentes, ela estará na AGM

• A_1 e A_2 são mínimas, então add $w(\{\{x, y\}\})$ também é mínima

é uma árvore geradora mínima

2. (2.5pts) Dado um grafo dirigido $G = (V, A)$ que corresponde a malha viária de uma cidade, considere um conjunto $R \subset A$. R é o potencial subconjunto de vias que seriam interditadas (removidas temporariamente da malha viária). Elabore um algoritmo (usando pseudocódigo ou alguma linguagem de programação) que receba o grafo G e o conjunto R e retorne o maior subconjunto possível de R no qual haja garantia de que há um caminho possível (de ida e de volta) entre qualquer par de vértices em G .

• identificar CFC $\rightarrow$ Kosaraju-Sharir

• entrada: $G, R \rightarrow$ Guloso

1. $R' \leftarrow \emptyset$

2. foreach $a \in R$

3. $A_{\text{teste}} \leftarrow A - R' - \{a\}$

4. $G_{\text{teste}} \leftarrow (V, A_{\text{teste}})$

5. $\text{Floresta_teste} \leftarrow \text{Kosaraju-Sharir}(G_{\text{teste}})$

6. $\text{Componentes} \leftarrow \text{contarComponentes}(\text{Floresta_teste})$

7. if $\text{Componentes} == 1$ then

8. $R' \leftarrow R' \cup \{a\}$

9. return R'

$\rightarrow$ P! cada are em R

$\left\{ \begin{array}{l} \text{Grafo de teste com arestas originais - ruas que já fechamos - rua em análise} \end{array} \right.$

$\left\{ \begin{array}{l} \text{Verificar se ainda é CFC;} \\ \text{se ainda só tiver uma componente,} \\ \text{add } \{a\} \text{ ao conjunto } R' \end{array} \right.$

• contar Componentes: $G_{\text{teste}} = (V, A_{\text{teste}})$, Floresta_teste

1. $\text{id_encontrados} \leftarrow \emptyset$

2. $\text{total_componentes} \leftarrow 0$

3. for each $v \in V$ do

4. $\text{id_componente} \leftarrow \text{Floresta_teste}[v]$

5. if $\text{id_componente} \neq \text{id_encontrado}$ then

6. $\text{id_encontrados} \leftarrow \text{id_encontrados} \cup \{\text{id_componente}\}$

7. $\text{total_componentes} \leftarrow \text{total_componentes} + 1$

8. return total_componentes

$\rightarrow$ conjunto $\tilde{v}$ aceitam repetição

$\rightarrow$ P! cada vértice

$\rightarrow$ se vértices estão na msm CFC, só tem 1 id

$\left\{ \begin{array}{l} \text{Se id for novo, add no encontrados e} \\ \text{incrementa no total} \end{array} \right.$

$\rightarrow$ se total = 1, toda a cidade é CFC

• entrada: G, R - backtracking

1. $\text{melhor_R}' \leftarrow \emptyset$

2. $x \leftarrow |R|$

3. indexar R como vetor: $R[1], R[2], \dots, R[x]$

4. $\text{buscar_combinações}(1, \emptyset, G, R, x)$

5. return $\text{melhor_R}'$

• $\text{buscar_combinações}(\text{id}_x, R'_{\text{atual}}, G, R, x)$

1. if $\text{id}_x > r$ then

2. $A_{\text{teste}} \leftarrow A - R'_{\text{atual}}$

3. $G_{\text{teste}} \leftarrow (V, A_{\text{teste}})$

4. $\text{Floresta_teste} \leftarrow \text{Kosaraju-Sharir}(G_{\text{teste}})$

5. if $\text{contarComponentes}(\text{Floresta_teste}) == 1$ then

6. if $|R'_{\text{atual}}| > |\text{melhor_R}'|$ then

7. $\text{melhor_R}' \leftarrow R'_{\text{atual}}$

8. return

9. $\text{buscar_combinações}(\text{id}_x + 1, R'_{\text{atual}} \cup \{R[\text{id}_x]\}, G, R, x)$

10. $\text{buscar_combinações}(\text{id}_x + 1, R'_{\text{atual}}, G, R, x)$

$\rightarrow$ Se $\text{id}_x > r$, já tem um subconjunto de teste completo

$\left\{ \begin{array}{l} \text{grafo teste sem arestas que foram} \\ \text{fechadas na rodada} \end{array} \right.$

$\left\{ \begin{array}{l} \text{se for só uma CFC} \end{array} \right.$

$\left\{ \begin{array}{l} \text{compara subconjunto atual com o último maior,} \\ \text{e se for maior, atualiza} \end{array} \right.$

$\rightarrow$ volta P! rua anterior e testa novos caminhos

$\rightarrow$ testa todas ramificações se a rua for interditada

$\rightarrow$ só desliza quando anterior não retorna, e testa as

ramificações se ficasse aberta

Se tivéssemos apenas 2 ruas ($R[1]$ e $R[2]$), a ordem exata em que o computador lê as linhas seria:

1. Rua 1: Executa a Linha 12 (Decide fechar a Rua 1). Pausa a Rua 1 e pula para a Rua 2.
2. Rua 2: Executa a Linha 12 (Decide fechar a Rua 2 também). Pausa a Rua 2 e pula para o idx 3.
3. idx 3: Bate na Linha 1 (Caso Base). Testa o Grafo sem a Rua 1 e sem a Rua 2. Executa a Linha 9 (return).
4. Rua 2: Acorda da pausa gerada pela Linha 12 e desce para a Linha 15 (Decide manter a Rua 2 aberta). Pula para o idx 3.
5. idx 3: Bate na Linha 1 (Caso Base). Testa o Grafo sem a Rua 1, mas com a Rua 2 aberta. Executa a Linha 9 (return).
6. Rua 2: Terminou suas opções, dá return automático e some da memória.
7. Rua 1: Acorda daquela primeira pausa da Linha 12 e finalmente desce para a Linha 15 (Decide manter a Rua 1 aberta).

→ G' add uma aresta $e = \{v_1, v_2\}$, onde peso w de e é menor que qualquer outra aresta do grafo original

• se for add uma nova aresta a uma AGM, um ciclo é criado;

↳ é preciso identificar o ciclo e remover a aresta mais pesada

↳ como e será a mais leve, com certeza estará na AGM

4. (2.5pts) Considere um grafo não-dirigido e ponderado $G = (V, E, w)$ e sua árvore geradora mínima $A \subseteq E$. Considere também o grafo $G' = (V, E \cup \{e\}, w')$ no qual $e = \{v_1, v_2\}$ e $w'(e) < w(\{u, v\})$ para qualquer $\{u, v\} \in E$. O algoritmo abaixo encontra uma **árvore geradora mínima** para G' ? Justifique sua resposta. *Não necessariamente*

```

Input :  $G = (V, E, w)$ ,  $e = \{v_1, v_2\}$ ,  $A$ 
1  $S' \leftarrow \{x \in N(v_1) : \{x, v_1\} \in A\}$ 
2  $S'' \leftarrow \{x \in N(v_2) : \{x, v_2\} \in A\}$ 
3  $v' \leftarrow \operatorname{argmax}_{x \in S'} \{w(\{v_1, x\})\}$ 
4  $v'' \leftarrow \operatorname{argmax}_{x \in S''} \{w(\{v_2, x\})\}$ 
5 if  $w(\{v_1, v'\}) > w(\{v_2, v''\})$  then
6    $R \leftarrow A - \{\{v_1, v'\}\}$ 
7 end
8 else
9    $R \leftarrow A - \{\{v_2, v''\}\}$ 
10 end
11  $R \leftarrow R \cup \{\{v_1, v_2\}\}$ 
12 return  $R$ 

```

comentários:

- 1, 2: conjuntos de vizinhos de v_1 e v_2 que estão conectados a ele pela AGM
- 3, 4: aresta + pesada dentre os vizinhos de v_1 e v_2
- 5-10: remove a + pesada e guarda em R
- 11: add a aresta leve e retorna

↳ assume que a aresta + pesada está conectada diretamente a v_1 ou v_2 , mas pode estar no meio do caminho

ex: $v_1 - \overset{2}{a} - \overset{10}{b} - \overset{3}{v_2}$ → $w(\{v_1, a\}) = 2$ } vai retirar a 3, add a 1,
 $w(\{v_2, b\}) = 3$ } mas a aresta 10 continuará → não será mínima

• é preciso olhar todo o caminho do novo ciclo, não só os adjacentes

```

1. caminhoArestas ← encontrar_caminho( $v_1, v_2, A$ )
2. aresta_max ← caminhoArestas[0]
3. for each  $e \in$  caminhoArestas do
4.   if  $w(e') > w(aresta\_max)$  then
5.      $aresta\_max \leftarrow e'$ 
6.  $R \leftarrow A - \{aresta\_max\}$ 
7.  $R \leftarrow R \cup \{e\}$ 
8. return  $R$ 

```

```

* encontrarCaminho(origem, destino, A)
1. visitados ← ∅
2. caminho ← 0
3. if DFS_caminho(origem, destino, A, visitados, caminho) then
4.   return caminho

```

* entra no grafo e add em caminho, se encontrar trecho sem saída, que não seja o destino, faz o backtracking


```
Função DFS_Caminho(atual, destino, A, visitados, caminho)
1 // Se chegamos ao objetivo, interrompe a busca com sucesso
2 Se atual == destino então
3 | return Verdadeiro
4 Fim Se
5
6 Adicione atual ao conjunto visitados
7
8 // Vasculha os vizinhos do vértice atual que estão conectados pela árvore A
9 Para cada aresta {atual, vizinho} em A faça
10 | Se vizinho não está em visitados então
11 | |
12 | | Adicione a aresta {atual, vizinho} na lista caminho
13 | |
14 | | // Avança recursivamente para o próximo vértice
15 | | Se DFS_Caminho(vizinho, destino, A, visitados, caminho) então
16 | | | return Verdadeiro // Se achou o destino lá na frente, propaga o sucesso
17 | | Fim Se
18 | |
19 | | // Backtracking: se o destino não foi encontrado por aqui,
20 | | // removemos esta aresta do caminho e tentamos o próximo vizinho
21 | | Remova a aresta {atual, vizinho} da lista caminho
22 | |
23 | Fim Se
24 Fim Para
25
26 return Falso // Retorna falso se explorou tudo por aqui e não achou o destino
```